CNNP-SKILL-2

THREE-POINT PACKET CAPTURE AND NETWORK PERFORMANCE ANALYSIS

Name: Usharani

Roll No.: 2520090049

To create a virtual network with Host1, a router and Host2; configure network connectivity and traffic conditions; capture packets at three points; and analyse packet headers, message content, delay, packet loss and TCP behaviour using command-line tools and Wireshark.

Step 1 – Create the network namespaces

Created three isolated Linux network namespaces named host1, router and host2. The namespaces represent the two end hosts and the intermediate router.

```
root@LAPTOP-1COJKKG6:~# sudo ip netns add host1
root@LAPTOP-1COJKKG6:~# sudo ip netns add router
root@LAPTOP-1COJKKG6:~# sudo ip netns add host2
root@LAPTOP-1COJKKG6:~#
```

Step 2 – Create virtual Ethernet links

Created virtual Ethernet interfaces to connect Host1 to the router and the router to Host2. A veth pair provides two connected interface endpoints, allowing the namespaces to communicate.

```
root@LAPTOP-1COJKKG6:~# sudo ip link add veth-h1 type veth peer name veth-r1

root@LAPTOP-1COJKKG6:~# sudo ip link add veth-r2 type veth peer name veth-h2
root@LAPTOP-1COJKKG6:~#
```

```
root@LAPTOP-1COJKKG6:~# sudo ip link set veth-h1 netns host1
root@LAPTOP-1COJKKG6:~# sudo ip link set veth-r1 netns router
root@LAPTOP-1COJKKG6:~# sudo ip link set veth-r2 netns router
root@LAPTOP-1COJKKG6:~# sudo ip link set veth-h2 netns host2
root@LAPTOP-1COJKKG6:~#
```

Step 3 – Configure Host1 and its interface

Moved the Host1-side interface into the host1 namespace, assigned its IP address and brought the interface up.

```
root@LAPTOP-1COJKKG6:~# sudo ip netns exec host1 ip addr add 10.0.1.1/30 dev
 veth-h1
root@LAPTOP-1COJKKG6:~# sudo ip netns exec host1 ip link set veth-h1 up
root@LAPTOP-1COJKKG6:~# sudo ip netns exec host1 ip link set lo up
root@LAPTOP-1COJKKG6:~#
```

Step 4 – Configure the router

Moved the router-side interfaces into the router namespace, assigned the router interface addresses and enabled IPv4 forwarding so packets could pass between the two network segments.

```
root@LAPTOP-1COJKKG6:~# sudo ip netns exec router ip addr add 10.0.1.2/30 de
v veth-r1
root@LAPTOP-1COJKKG6:~# sudo ip netns exec router ip addr add 10.0.2.1/30 de
v veth-r2
root@LAPTOP-1COJKKG6:~# sudo ip netns exec router ip link set veth-r1 up
root@LAPTOP-1COJKKG6:~# sudo ip netns exec router ip link set veth-r2 up
root@LAPTOP-1COJKKG6:~# sudo ip netns exec router ip link set lo up
root@LAPTOP-1COJKKG6:~#
```

Step 5 – Configure Host2

Moved the Host2-side interface into the host2 namespace, assigned its IP address and brought the interface up.

```
root@LAPTOP-1COJKKG6:~# sudo ip netns exec host2 ip addr add 10.0.2.2/30 dev
 veth-h2
sudo ip netns exec host2 ip link set veth-h2 up
sudo ip netns exec host2 ip link set lo up
root@LAPTOP-1COJKKG6:~#
```

Step 6 – Add routing information

Configured routes on the end hosts so traffic for the opposite subnet is sent through the router. This establishes the routed path from Host1 to Host2.

```
root@LAPTOP-1COJKKG6:~# sudo ip netns exec router sysctl -w net.ipv4.ip_forw
ard=1
sudo ip netns exec host1 ip route add 10.0.2.0/30 via 10.0.1.2
sudo ip netns exec host2 ip route add 10.0.1.0/30 via 10.0.2.1
net.ipv4.ip_forward = 1
root@LAPTOP-1COJKKG6:~#
```

Step 7 – Test end-to-end connectivity

Used ping from Host1 to the Host2 address (10.0.2.2) to verify that packets could travel across the virtual network and receive replies.

```
root@LAPTOP-1COJKKG6:~# sudo ip netns exec host1 ping -c 3 10.0.2.2
PING 10.0.2.2 (10.0.2.2) 56(84) bytes of data.
64 bytes from 10.0.2.2: icmp_seq=1 ttl=63 time=0.032 ms
64 bytes from 10.0.2.2: icmp_seq=2 ttl=63 time=0.047 ms
64 bytes from 10.0.2.2: icmp_seq=3 ttl=63 time=0.042 ms

--- 10.0.2.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2000ms
rtt min/avg/max/mdev = 0.032/0.040/0.047/0.006 ms
root@LAPTOP-1COJKKG6:~#
```

Step 8 – Configure network delay and packet loss

Applied traffic-control netem settings on the relevant router interface to introduce network impairments for the experiment. The later measurements showed an approximately 20 ms one-way delay and a packet-loss observation during the MTR test.

```
root@LAPTOP-1COJKKG6:~# sudo ip netns exec router tc qdisc add dev veth-r2 r
oot netem delay 20ms loss 1%
root@LAPTOP-1COJKKG6:~#
```

```
root@LAPTOP-1COJKKG6:~# sudo ip netns exec router tc qdisc show dev veth-r2
qdisc netem 8001: root refcnt 13 limit 1000 delay 20ms loss 1%
root@LAPTOP-1COJKKG6:~#
```

Step 9 – Start packet capture at Host1

Started tcpdump on the Host1-facing virtual interface and saved the captured traffic to /tmp/cap1.pcap.

```
root@LAPTOP-1COJKKG6:~# sudo ip netns exec host1 tshark -i veth-h1 -w /tmp/c
ap1.pcap
Running as user "root" and group "root". This could be dangerous.
Capturing on 'veth-h1'
```

```
Terminal 2
root@LAPTOP-1COJKKG6:~# sudo ip netns exec router tshark -i veth-r1 -w /tmp/cap2.pcap
Running as user "root" and group "root". This could be dangerous.
Capturing on 'veth-r1'
```

Step 10 – Start packet capture at the router

Started a second packet capture on the router-facing interface and saved the traffic to /tmp/cap2.pcap.

```
Terminal 3
root@LAPTOP-1COJKKG6:~# sudo ip netns exec host2 tshark -i veth-h2 -w /tmp/cap3.pcap
Running as user "root" and group "root". This could be dangerous.
Capturing on 'veth-h2'
```

Step 11 – Start packet capture at Host2

Started packet capture on the Host2-facing interface and saved the traffic to /tmp/cap3.pcap. Capturing at all three points makes it possible to compare the same traffic as it moves through the network.

```
Terminal 2
root@LAPTOP-1COJKKG6:~# sudo ip netns exec router tshark -i veth-r1 -w /tmp/cap2.pcap
Running as user "root" and group "root". This could be dangerous.
Capturing on 'veth-r1'
17
```

```
Terminal 3
root@LAPTOP-1COJKKG6:~# sudo ip netns exec host2 tshark -i veth-h2 -w /tmp/cap3.pcap
Running as user "root" and group "root". This could be dangerous.
Capturing on 'veth-h2'
17
```

Step 12 – Send a TCP test message

Used netcat with TCP port 1234 to receive and send a test message. The captured payload included the lines “Hello host!”, “This is Alekhya”, “Computer Networks Lab” and “Three Point Capture Experiment”.

```
root@LAPTOP-1COJKKG6:~# sudo ip netns exec host2 nc -l 1234
Hello host!
This is Alekhya
Computer Networks Lab
Three Point Capture Experiment
```

.

```
Capturing on 'veth-h1'
17
```

Step 13 – Check and preserve the capture files

Changed the permissions of the three pcap files so they could be read and inspected, then listed their sizes to confirm that the files existed.

```
root@LAPTOP-1COJKKG6:~# sudo chmod 644 /tmp/cap1.pcap /tmp/cap2.pcap /tmp/cap3.pcap
root@LAPTOP-1COJKKG6:~# ls -lh /tmp/cap1.pcap /tmp/cap2.pcap /tmp/cap3.pcap
-rw-r--r-- 1 root root 2.1K Nov  7 07:39 /tmp/cap1.pcap
-rw-r--r-- 1 root root 2.1K Nov  7 07:39 /tmp/cap2.pcap
-rw-r--r-- 1 root root 2.0K Nov  7 07:39 /tmp/cap3.pcap
root@LAPTOP-1COJKKG6:~#
```

Step 14 – Analyse IP TTL values

Used tshark to extract the IPv4 TTL field from each capture. The observed values were 63 and 64. For the Host2-directed packets from 10.0.1.1 to 10.0.2.2 in cap3.pcap, the displayed TTL was 63.

```
root@LAPTOP-1COJKKG6:~# tshark -r /tmp/cap1.pcap -T fields -e ip.ttl -Y "ip" 2>/dev/null | sort | uniq -c
      4 63
      5 64
root@LAPTOP-1COJKKG6:~# tshark -r /tmp/cap2.pcap -T fields -e ip.ttl -Y "ip" 2>/dev/null | sort | uniq -c
      4 63
      5 64
root@LAPTOP-1COJKKG6:~# tshark -r /tmp/cap3.pcap -T fields -e ip.ttl -Y "ip" 2>/dev/null | sort | uniq -c
      5 63
      4 64
root@LAPTOP-1COJKKG6:~#
```

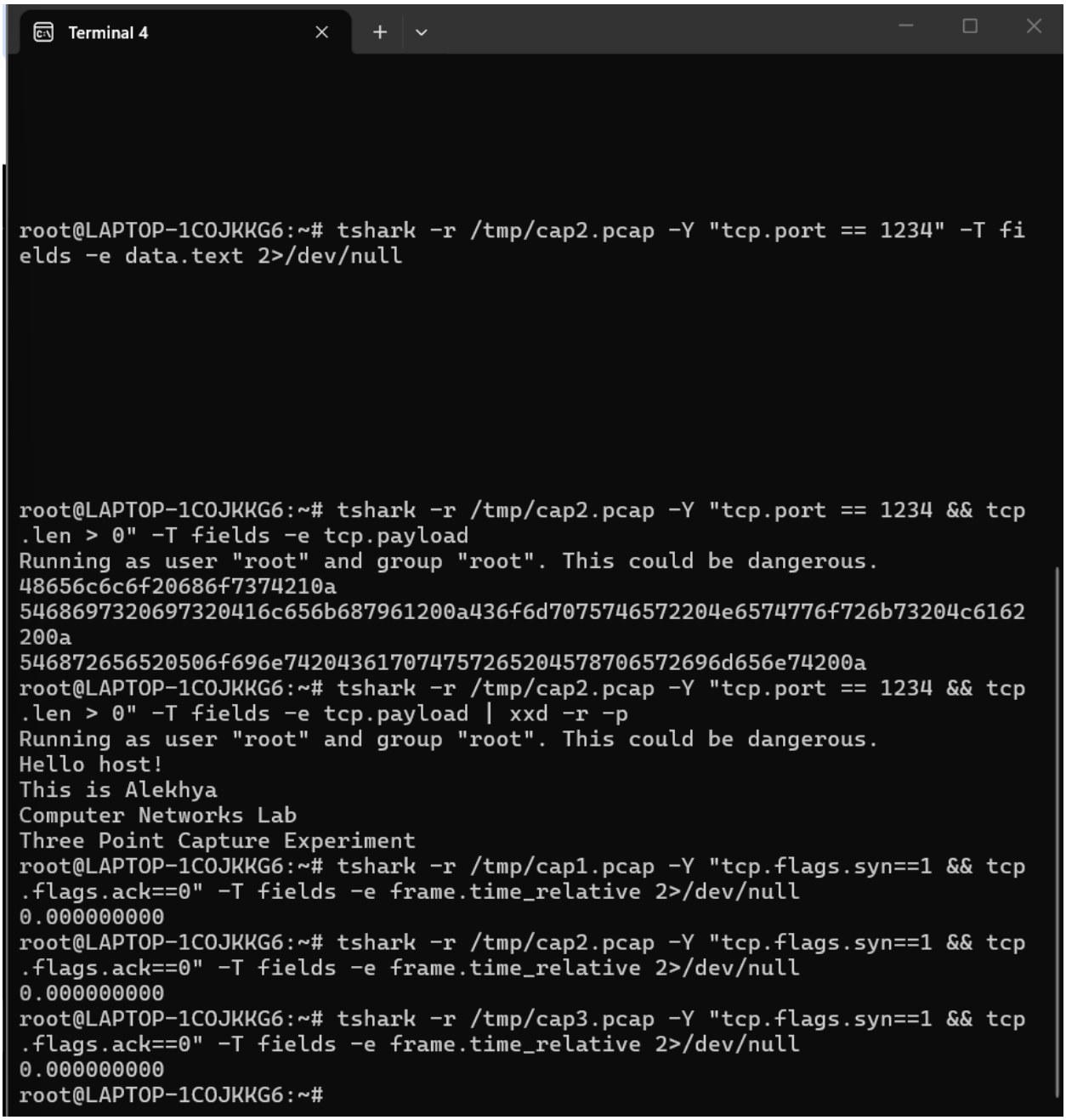
Step 15 – Extract the TCP message payload

Used a display filter for TCP port 1234 and non-empty TCP segments. The tcp.payload field was extracted as hexadecimal and converted back to readable text. The original test message was successfully recovered from the capture.

```
root@LAPTOP-1COJKKG6:~# tshark -r /tmp/cap2.pcap -Y "tcp.port == 1234 && tcp.len > 0" -T fields -e tcp.payload
Running as user "root" and group "root". This could be dangerous.
48656c6c6f20686f7374210a
546869732069732041 6c656b687961200a436f6d7075746572204e6574776f726b73204c6162200a
5468726565206f696e74204361707475726520457870657269 6d656e74200a
root@LAPTOP-1COJKKG6:~# tshark -r /tmp/cap2.pcap -Y "tcp.port == 1234 && tcp.len > 0" -T fields -e tcp.payload | xxd -r
-p
Running as user "root" and group "root". This could be dangerous.
Hello host!
This is Alekhya
Computer Networks Lab
Three Point Capture Experiment
root@LAPTOP-1COJKKG6:~#
```

Step 16 – Compare packet timestamps

Extracted frame.time_epoch timestamps for TCP data packets from cap1.pcap and cap3.pcap. For the first corresponding packet, the timestamp difference was about 0.02108 seconds (approximately 21.08 ms), which is close to the configured delay. This comparison assumes the capture clocks are sufficiently aligned.



```
root@LAPTOP-1COJKKG6:~# tshark -r /tmp/cap1.pcap -Y "tcp.port == 1234 && tcp
.len > 0" -T fields -e frame.time_epoch
\Running as user "root" and group "root". This could be dangerous.
1794036818.649871810
1794036857.536586235
1794036857.557728521
root@LAPTOP-1COJKKG6:~# tshark -r /tmp/cap3.pcap -Y "tcp.port == 1234 && tcp
.len > 0" -T fields -e frame.time_epoch
Running as user "root" and group "root". This could be dangerous.
1794036818.670951792
1794036857.557582316
1794036857.578912910
root@LAPTOP-1COJKKG6:~#
```

.

Step 17 – Check TCP retransmission indications

Used the tshark display filter tcp.analysis.retransmission on each individual pcap file. The command returned zero matching packets for cap1.pcap, cap2.pcap and cap3.pcap. This means no retransmissions were identified by that filter in those individual files; it does not prove that packet loss never occurred.

```
root@LAPTOP-1COJKKG6:~# tshark -r /tmp/cap1.pcap -Y "tcp.analysis.retransmis
sion" 2>/dev/null | wc -l
0
root@LAPTOP-1COJKKG6:~# tshark -r /tmp/cap2.pcap -Y "tcp.analysis.retransmis
sion" 2>/dev/null | wc -l
0
root@LAPTOP-1COJKKG6:~# tshark -r /tmp/cap3.pcap -Y "tcp.analysis.retransmis
sion" 2>/dev/null | wc -l
0
root@LAPTOP-1COJKKG6:~#
```

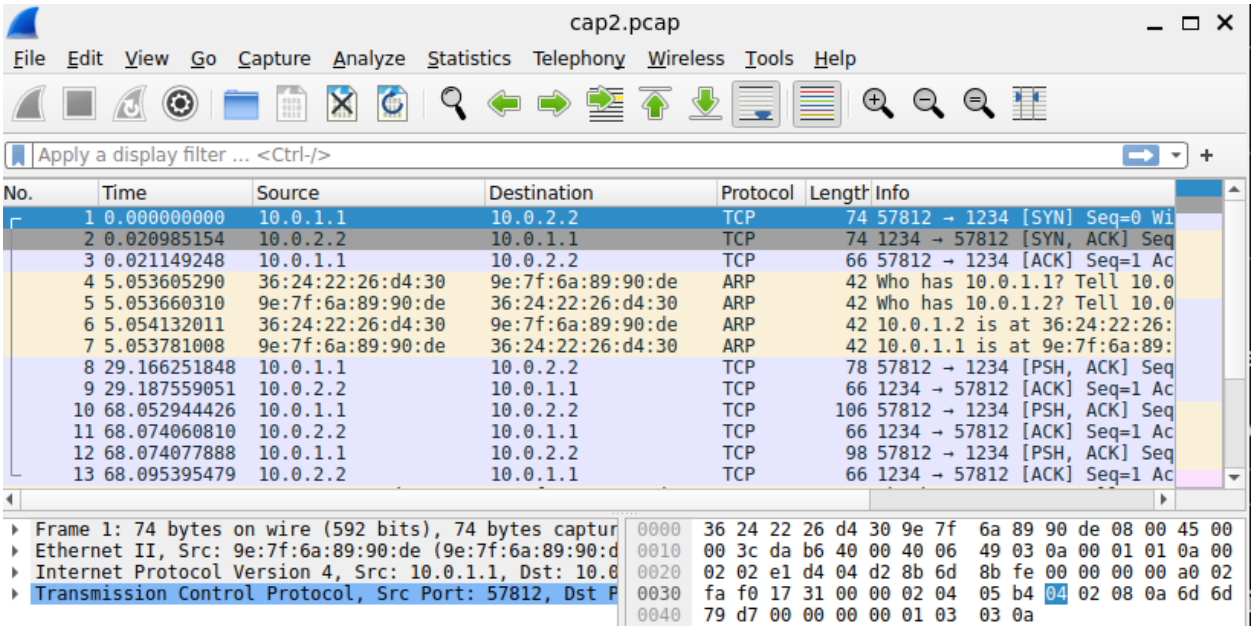
Step 18 – Measure RTT and packet loss with MTR

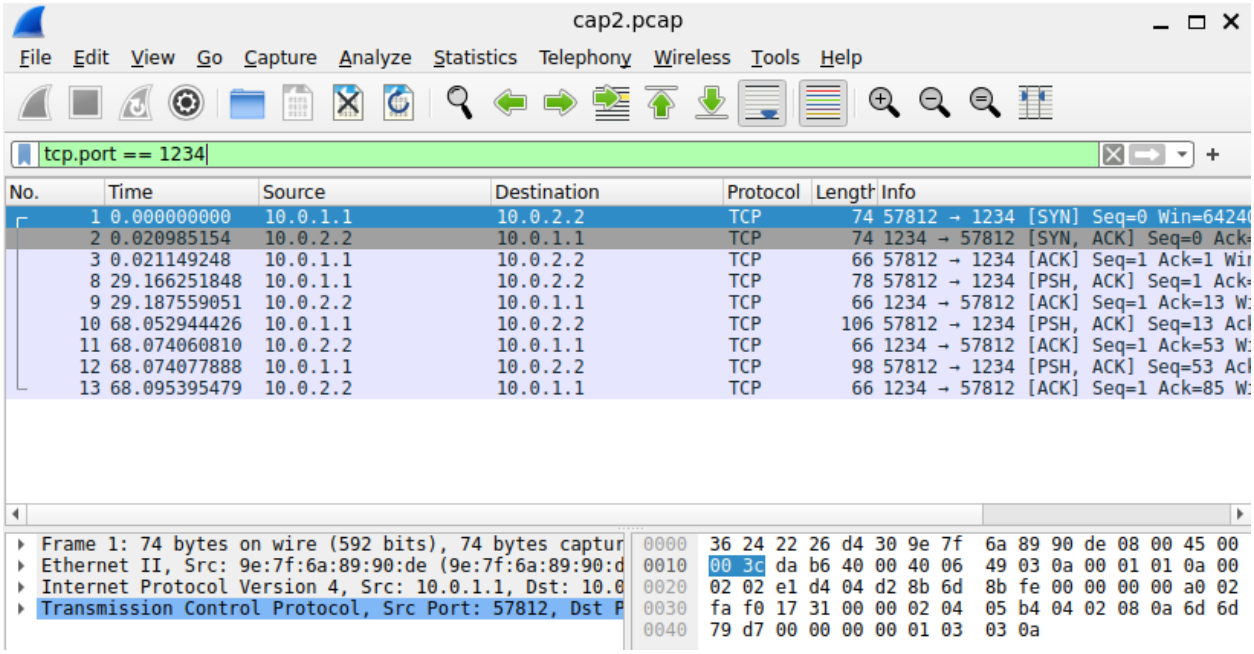
Ran MTR from the host1 namespace to 10.0.2.2. The report showed 0.0% loss and an average RTT of 0.2 ms at the router hop (10.0.1.2), and 10.0% loss with an average RTT of 21.3 ms at Host2 (10.0.2.2). The test used 10 probes, so the observed percentage is a small-sample result.

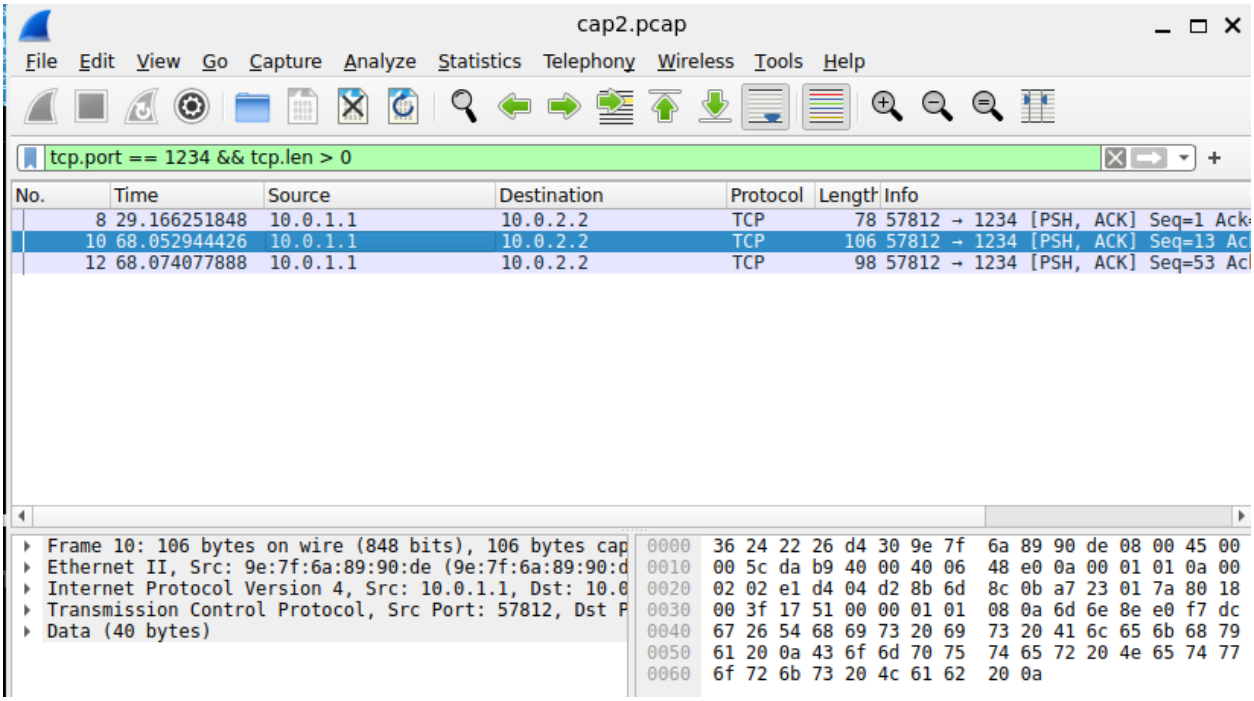
```
root@LAPTOP-1COJKKG6:~# sudo ip netns exec host1 mtr --report 10.0.2.2
Start: 2026-11-07T07:51:44+0000
HOST: LAPTOP-1COJKKG6             Loss%   Snt   Last   Avg  Best  Wrst StDev
  1.|-- 10.0.1.2                   0.0%    10    0.1   0.2   0.0   1.1   0.3
  2.|-- 10.0.2.2                  10.0%    10   21.2  21.3  21.0  22.0   0.3
root@LAPTOP-1COJKKG6:~#
```

Step 19 – Inspect packets in Wireshark

Opened cap2.pcap in Wireshark and applied the display filter tcp.port == 1234. Selected a TCP data packet and inspected the Frame, Ethernet II, IPv4, TCP and Data sections. The payload bytes were visible in the packet-bytes pane.

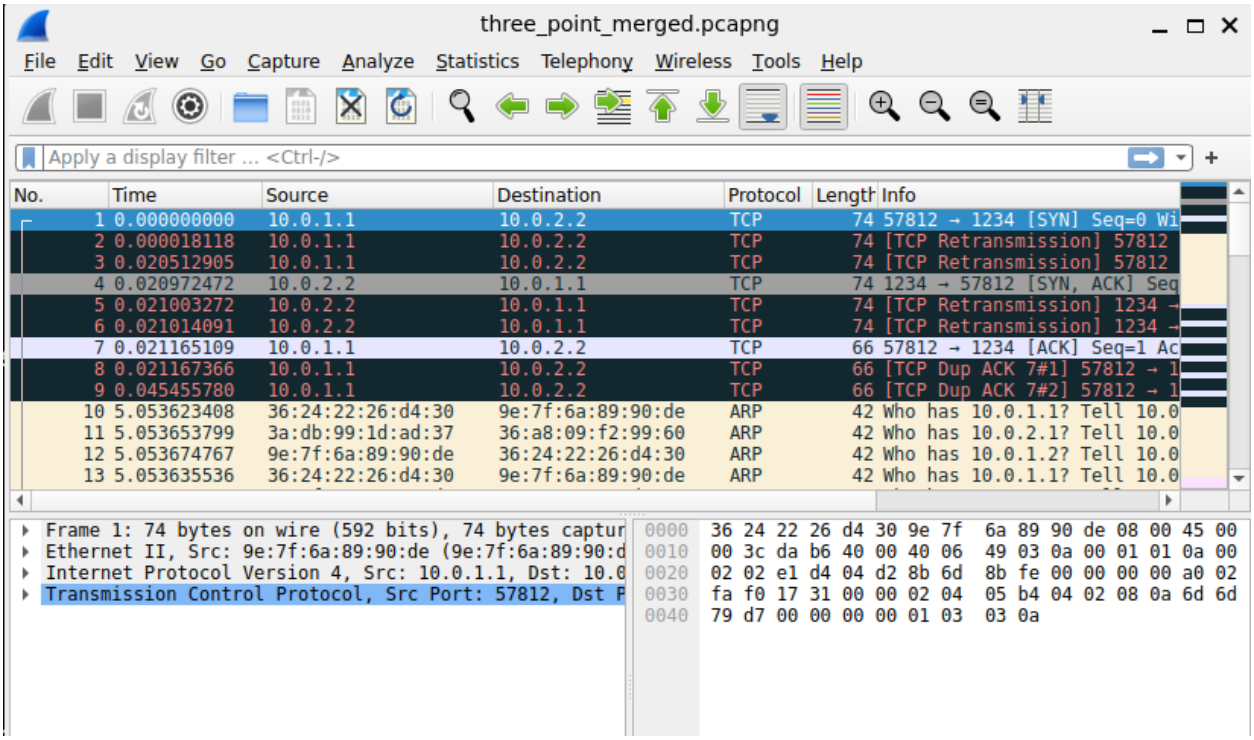

cap2.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000000 10.0.1.1 10.0.2.2 TCP 74 57812 → 1234 [SYN] Seq=0 Wi
2 0.020985154 10.0.2.2 10.0.1.1 TCP 74 1234 → 57812 [SYN, ACK] Seq
3 0.021149248 10.0.1.1 10.0.2.2 TCP 66 57812 → 1234 [ACK] Seq=1 Ac
4 5.053605290 36:24:22:26:d4:30 9e:7f:6a:89:90:de ARP 42 Who has 10.0.1.1? Tell 10.0
5 5.053660310 9e:7f:6a:89:90:de 36:24:22:26:d4:30 ARP 42 Who has 10.0.1.2? Tell 10.0
6 5.054132011 36:24:22:26:d4:30 9e:7f:6a:89:90:de ARP 42 10.0.1.2 is at 36:24:22:26:
7 5.053781008 9e:7f:6a:89:90:de 36:24:22:26:d4:30 ARP 42 10.0.1.1 is at 9e:7f:6a:89:
8 29.166251848 10.0.1.1 10.0.2.2 TCP 78 57812 → 1234 [PSH, ACK] Seq
9 29.187559051 10.0.2.2 10.0.1.1 TCP 66 1234 → 57812 [ACK] Seq=1 Ac
10 68.052944426 10.0.1.1 10.0.2.2 TCP 106 57812 → 1234 [PSH, ACK] Seq
11 68.074060810 10.0.2.2 10.0.1.1 TCP 66 1234 → 57812 [ACK] Seq=1 Ac
12 68.074077888 10.0.1.1 10.0.2.2 TCP 98 57812 → 1234 [PSH, ACK] Seq
13 68.095395479 10.0.2.2 10.0.1.1 TCP 66 1234 → 57812 [ACK] Seq=1 Ac
Frame 1: 74 bytes on wire (592 bits), 74 bytes captur
Ethernet II, Src: 9e:7f:6a:89:90:de (9e:7f:6a:89:90:d
Internet Protocol Version 4, Src: 10.0.1.1, Dst: 10.0
Transmission Control Protocol, Src Port: 57812, Dst P
0000 36 24 22 26 d4 30 9e 7f 6a 89 90 de 08 00 45 00
0010 00 3c da b6 40 00 40 06 49 03 0a 00 01 01 0a 00
0020 02 02 e1 d4 04 d2 8b 6d 8b fe 00 00 00 00 a0 02
0030 fa f0 17 31 00 00 02 04 05 b4 04 02 08 0a 6d 6d
0040 79 d7 00 00 00 00 01 03 03 0a


cap2.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp.port == 1234
No. Time Source Destination Protocol Length Info
1 0.000000000 10.0.1.1 10.0.2.2 TCP 74 57812 → 1234 [SYN] Seq=0 Win=64240
2 0.020985154 10.0.2.2 10.0.1.1 TCP 74 1234 → 57812 [SYN, ACK] Seq=0 Ack
3 0.021149248 10.0.1.1 10.0.2.2 TCP 66 57812 → 1234 [ACK] Seq=1 Ack=1 Wi
8 29.166251848 10.0.1.1 10.0.2.2 TCP 78 57812 → 1234 [PSH, ACK] Seq=1 Ack
9 29.187559051 10.0.2.2 10.0.1.1 TCP 66 1234 → 57812 [ACK] Seq=1 Ack=13 W
10 68.052944426 10.0.1.1 10.0.2.2 TCP 106 57812 → 1234 [PSH, ACK] Seq=13 Ac
11 68.074060810 10.0.2.2 10.0.1.1 TCP 66 1234 → 57812 [ACK] Seq=1 Ack=53 W
12 68.074077888 10.0.1.1 10.0.2.2 TCP 98 57812 → 1234 [PSH, ACK] Seq=53 Ac
13 68.095395479 10.0.2.2 10.0.1.1 TCP 66 1234 → 57812 [ACK] Seq=1 Ack=85 W
Frame 1: 74 bytes on wire (592 bits), 74 bytes captur
Ethernet II, Src: 9e:7f:6a:89:90:de (9e:7f:6a:89:90:d
Internet Protocol Version 4, Src: 10.0.1.1, Dst: 10.0
Transmission Control Protocol, Src Port: 57812, Dst P
0000 36 24 22 26 d4 30 9e 7f 6a 89 90 de 08 00 45 00
0010 00 3c da b6 40 00 40 06 49 03 0a 00 01 01 0a 00
0020 02 02 e1 d4 04 d2 8b 6d 8b fe 00 00 00 00 a0 02
0030 fa f0 17 31 00 00 02 04 05 b4 04 02 08 0a 6d 6d
0040 79 d7 00 00 00 00 01 03 03 0a



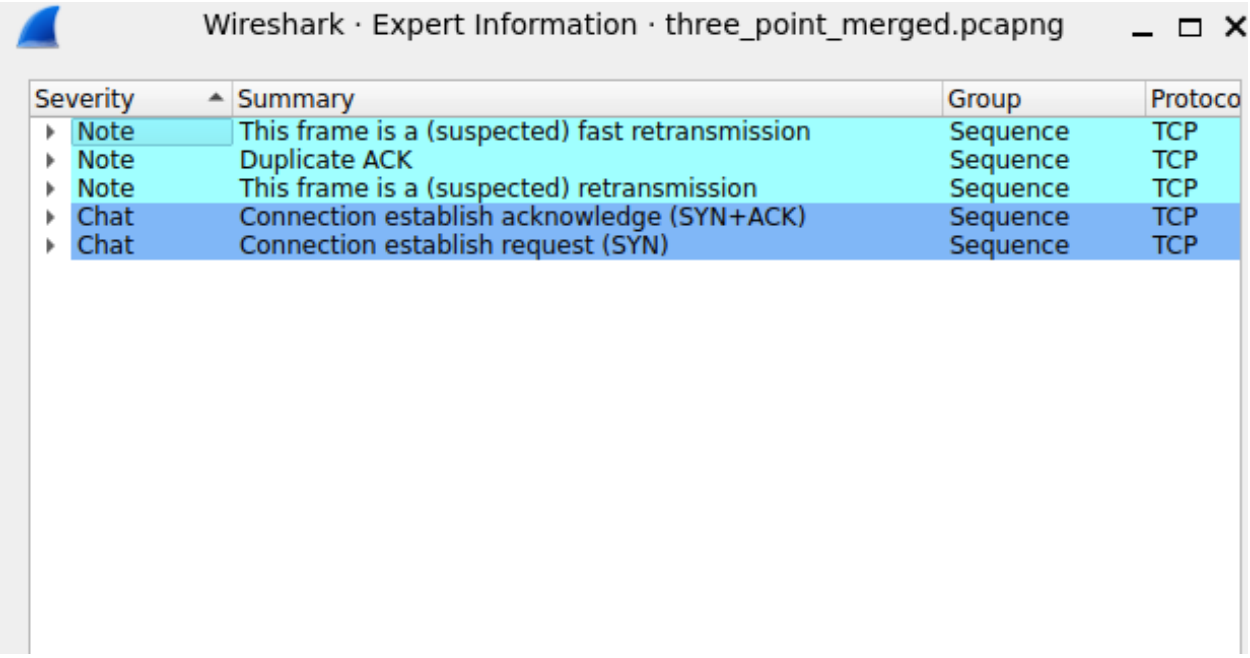
Step 20 – Merge the three packet captures

Used mergecap to combine cap1.pcap, cap2.pcap and cap3.pcap into /tmp/three_point_merged.pcapng. capinfos verified 53 packets across three interfaces: 18 packets on veth-h1, 18 on veth-r1 and 17 on veth-h2.



Step 21 – Review Wireshark Expert Information

Opened Analyze → Expert Information for the merged capture. Wireshark displayed suspected fast retransmission, duplicate ACK and suspected retransmission notes. Because the file combines observations from multiple capture interfaces, repeated observations of the same packet can produce analysis indications; these notes should not be treated alone as proof of end-to-end retransmission.



Step 22 – Clean up the network namespaces

Deleted the host1, router and host2 network namespaces using ip netns del. Ran ip netns list afterward; it returned no listed namespaces, confirming that the experiment namespaces were removed.